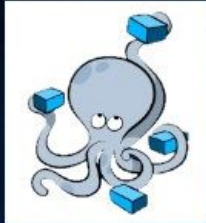




What is Docker Compose?

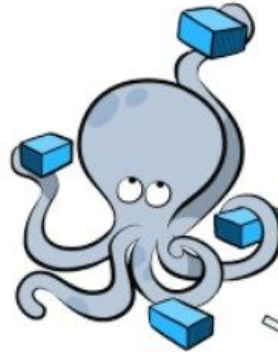
- **Docker Compose** is a tool that assists in defining and sharing multi-container applications. By using Compose, we can define the services in a YAML file, as well as spin them up and tear them down with one single command.
- Benefits of Docker Compose
 - Single host deployment - This means you can run everything on a single piece of hardware
 - Quick and easy configuration - Due to YAML scripts
 - High productivity - Docker Compose reduces the time it takes to perform tasks
 - Security - All the containers are isolated from each other, reducing the threat landscape

What is Docker Compose?



Docker Compose

```
docker-compose.yml
version: "3.7"
services:
  db:
    image: mysql:8.0.19
    restart: always
    environment:
      - MYSQL_DATABASE=example
      - MYSQL_ROOT_PASSWORD=password
  app:
    build: app
    restart: always
  web:
    build: web
    restart: always
    ports:
      - 80:80
```

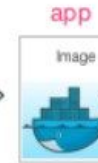


Dockerfile

Dockerfile



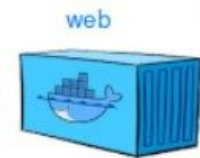
run



run



run



Run multiple containers as a service from 1 .yaml file

Docker Compose Installation

1. Install: <https://docs.docker.com/compose/install/other/>

2. `$ sudo curl -SL`

`https://github.com/docker/compose/releases/download/v2.12.2/docker-compose-linux-x86_64 -o /usr/local/bin/docker-compose`

3. `$ sudo ln -s /usr/local/bin/docker-compose /usr/bin/docker-compose`

What is YAML?

- **YAML** Ain't Markup Language is a data serialization language(configuration file) that matches user's expectations about data. It designed to be human friendly and works perfectly with other programming languages and use extension **.yaml** or **.yml**.

YAML	XML	JSON
<pre>apis: - name: login port: 8080 - name: profile port: 8090</pre>	<pre><apis> <api> <name>login</name> <port>8080</port> </api> <api> <name>profile</name> <port>8090</port> </api> </apis></pre>	<pre>{ "apis": [{ "name": "login", "port": 8080 }, { "name": "profile", "port": 8090 }] }</pre>

Docker Compose Structure in YAML

```
---  
version: "3.8"  
services:  
...  
volumes:  
...  
networks:  
...
```

- \$ docker compose up -d
- \$ docker compose up -d --build # Always build or rebuild image
- \$ docker compose -f other-compose-name.yaml up -d

Deploy NGINX

- **always**: always pull
- **never**: don't pull (breaks if the image can not be found)
- **missing**: pulls if the image is not cached (default)
- **build**: always build or rebuild

```
$ lscpu | grep -E '^CPU\(s\):|Core|Socket'
$ docker container inspect my-nginx | grep -i cpu
# NOTE: 1 Cpus = NanoCpus x 10-9
```

```
version: '3.8'
services:
  nginx-web:
    container_name: "my-nginx"
    image: xeng/custom-nginx:1.1
    pull_policy: always
    restart: always
    ports:
      - "85:80"
    deploy:
      resources:
        limits:
          cpus: '1.5'
          memory: '4g'
        reservations:
          cpus: '0.5'
          memory: '1g'
```

Deploy NGINX

- `$ docker compose -f nginx-compose.yaml up -d --build`
- `$ docker compose -f nginx-compose.yaml down`
- `$ docker compose -f nginx-compose.yaml start`
- `$ docker compose -f nginx-compose.yaml stop`
- `$ docker compose -f nginx-compose.yaml pause`
- `$ docker compose -f nginx-compose.yaml unpause`
- `$ docker compose -f nginx-compose.yaml ps`

Deploy Wordpress with Mysql

```
version: "3.8"
services:
  mysql:
    container_name: "new-mysql-db"
    image: mysql
    ports:
      - "3308:3306"
    environment:
      - MYSQL_ROOT_PASSWORD=123
      - MYSQL_USER=dara
      - MYSQL_PASSWORD=123
      - MYSQL_DATABASE=test
```

```
volumes:
  - new-mysql:/var/lib/mysql
networks:
  - new-mysql-net
wordpress:
  depends_on:
    - mysql
  container_name: "my-wordpress"
  image: wordpress
  ports:
    - "81:80"
```

Deploy Wordpress with Mysql

environment:

- WORDPRESS_DB_HOST=mysql
- WORDPRESS_DB_USER=dara
- WORDPRESS_DB_PASSWORD=123
- WORDPRESS_DB_NAME=test

volumes:

- wordpress-db:/var/www/html

networks:

- new-mysql-net

volumes:

new-mysql:

wordpress-db:

networks:

new-mysql-net:

name: new-mysql-network

driver: bridge

Deploy Spring, Postgres and Reactjs

```
version: "3.8"
services:
  postgres-db:
    container_name: "postgres-db-cont"
    image: postgres:13.8
    ports:
      - "5433:5432"
    environment:
      - POSTGRES_PASSWORD=123
    volumes:
      - pg-db:/var/lib/postgresql/data
    networks:
      - my-net
```

```
spring-api:
  depends_on:
    - postgres-db
  container_name: "spring-api-cont"
  build:
    context: .
    dockerfile: ./spring-dockerfile
  ports:
    - "8081:8080"
  volumes:
    - app-files:/src/main/resources/images
  networks:
    - my-net
```

Deploy Spring, Postgres and Reactjs

```
react-ui:
  depends_on:
    - spring-api
  container_name: "react-ui-cont"
  build:
    context: ./seng-react-test
    dockerfile: Dockerfile
  ports:
    - "3001:3000"
  networks:
    - my-net
volumes:
  pg-db:
  app-files:
networks:
  my-net:
```

```
# Build in Current Context with Dockerfile
---
build: .
---
```

How to auto backup Postgresql Database

- `$ crontab -e`
 - `*/2 * * * * docker exec f4118fb4c7f9 pg_dump -U postgres postgres > /home/seng/backup-db/pg-backup-$(date +%F@%T).sql`
- `*/2 * * * *` : Schedule date and time in crontab
- `docker exec` : Execute into container without interact in it
- `f4118fb4c7f9` : Use container name or id
- `pg_dump -U postgres postgres`: Backup database “postgres” for user -U “postgres”
- `> /home/seng/backup-db/pg-backup-$(date +%F@%T).sql` : Write result into text

A large, faint watermark of the ISTAD logo is centered in the background. It is a circular emblem with a blue outer ring containing the text 'INSTITUTE OF SCIENCE AND TECHNOLOGY ADVANCED DEVELOPMENT' and Khmer text. Inside the ring is a blue circle with a white atomic symbol. The word 'ISTAD' is written in red across the bottom of the inner circle. Binary code '01010100 01000001 01000000' is visible along the top inner edge of the ring.

Thank you

Begin with the theory, Start with the practical